

HiPREX: Hierarchical Planner–Recursive Executor

Ajay Suthaharan

January 1, 2026

Section 1: Background & Motivation

Recent advances in artificial intelligence have been largely driven by model scaling. In particular, large language models (LLMs) have demonstrated that increasing parameter counts, training data volume, and computational resources can yield substantial improvements in language understanding, reasoning, and generalization. This scaling paradigm has enabled models to perform complex tasks, such as multi-step reasoning, code generation, and natural language planning, using a single forward inference pass.

However, the continued growth of model size has introduced significant practical challenges, especially during inference. While training large models is resource-intensive, inference imposes persistent constraints on deployment environments. Modern LLMs require substantial memory not only to store model parameters, but also to maintain intermediate activations and attention key-value caches across long contexts. As a result, memory consumption and bandwidth have emerged as critical bottlenecks, often surpassing raw compute as the dominant factor influencing cost, latency, and deployability.

These memory constraints are particularly limiting in real-time systems, edge devices, and cost-sensitive applications, where available RAM is fixed or severely constrained. Even in data-center environments, inference memory requirements scale linearly with model size and context length, leading to increased infrastructure costs and reduced scalability. Consequently, the prevailing approach of parameter-centric scaling exhibits diminishing practical returns, as marginal gains in model capability come at disproportionate memory overhead.

Moreover, most contemporary neural architectures rely on static inference graphs, in which a fixed computation is performed regardless of task complexity. This design assumes that all reasoning must be encoded implicitly within model parameters and executed in a single pass. Such an approach contrasts with classical algorithmic systems, where complex behavior is often achieved through repeated application of simple operations over time, rather than through large static state representations.

These observations motivate the exploration of alternative architectural paradigms that trade static model scale for temporal computation. By reusing a small set of parameters across multiple recursive or iterative steps, it becomes possible to incrementally refine internal representations while maintaining bounded memory usage. In this hierarchical perspective, high-capacity models can be leveraged for coarse semantic understanding, while smaller, reusable components perform structured refinement, verification, or reasoning over time.

This report explores this design space and argues that hierarchical and recursive computation provides a promising path toward memory-efficient intelligent systems. By explicitly separating semantic initialization from iterative reasoning, such architectures aim to preserve the expressive power of large models while mitigating the memory constraints that increasingly dominate modern AI deployment.

Section 2: HiPREX Architecture

2.1 Design Philosophy

Modern neural architectures exhibit fundamentally different computational strengths. Transformer-based models excel at global contextual reasoning, enabling rich semantic understanding across long sequences. Recurrent Neural Networks (RNNs) operate through discrete state updates, making them suitable for step-by-step processing. State Space Models (SSMs) provide a complementary capability by maintaining continuous, persistent memory over time. Each of these paradigms captures a distinct aspect of intelligent computation.

However, contemporary systems typically commit to a single paradigm throughout inference. As a result, all reasoning is forced into a uniform computational mode, regardless of task structure or complexity. This rigidity contributes to inefficiencies, particularly in memory usage, as global attention mechanisms are applied even when localized or iterative computation would suffice.

HiPREX is motivated by the observation that intelligent reasoning often requires **multiple modes of computation**, applied at different stages of a task. Rather than replacing existing neural architectures, HiPREX introduces a hierarchical control framework that determines *when* global reasoning, discrete updates, or continuous memory should be employed.

2.2 High-Level Architecture

HiPREX is a hierarchical inference system composed of three primary components:

1. **Semantic Initializer (Global Reasoning Module)**
2. **Recursive Reasoning Core (Discrete Update Module)**
3. **Halting and Control Mechanism**

The system executes in stages. User input is first processed by the Semantic Initializer, which performs global contextual reasoning to extract meaning, intent, and high-level structure. This initial representation is then passed to the Recursive Reasoning Core, which iteratively refines the internal state through repeated, discrete updates. Throughout execution, the Halting and Control Mechanism governs the flow of computation, determining when recursion should continue or terminate.

By explicitly separating global reasoning from iterative refinement, HiPREX avoids repeatedly invoking memory-intensive operations when they are no longer required.

2.3 Semantic Initializer: Global Contextual Reasoning

The Semantic Initializer is implemented using a transformer-based large language model. Its role is to perform **global reasoning** over the input, leveraging self-attention to capture long-range dependencies and semantic relationships. This component translates unstructured input into a structured internal state that encodes goals, constraints, and relevant concepts.

Crucially, the Semantic Initializer is executed only once per request. After producing the initial state, it exits the inference loop. This prevents unbounded growth in attention-related memory structures, such as key–value caches, which would otherwise scale with sequence length and inference duration.

In the HiPREX framework, transformers are used precisely where they are strongest: **initial global understanding**, not prolonged iterative computation.

2.4 Recursive Reasoning Core

The Recursive Reasoning Core is a lightweight model designed for repeated execution. Conceptually similar to a recurrent neural network, this component applies a fixed transformation to the current internal state at each step, producing a refined state for the next iteration.

Unlike transformer-based inference, the Recursive Reasoning Core operates using **discrete state updates** and reuses the same parameters at every step. As a result, memory usage remains constant across iterations, and computational complexity is expressed in time rather than space. This allows the system to perform extended reasoning, verification, or optimization without accumulating large intermediate representations.

Where appropriate, the internal state may incorporate structured or continuous representations inspired by State Space Models, enabling persistent memory without reliance on attention-based mechanisms.

2.5 Halting and Control Mechanism

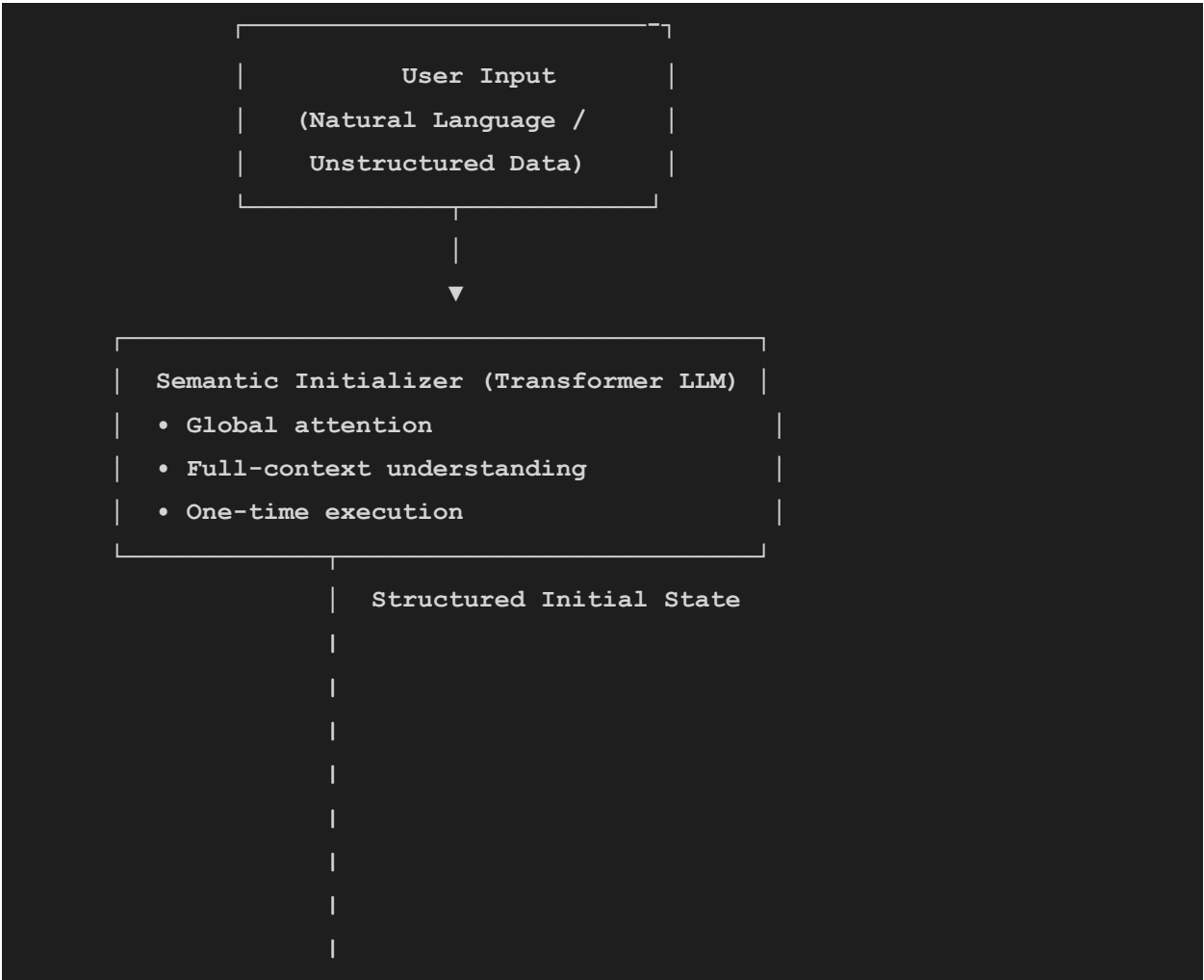
A defining feature of HiPREX is its explicit control logic. The Halting and Control Mechanism determines when each computational mode is active and when recursive execution should terminate. Halting decisions may be based on convergence of the internal state, confidence estimates, or predefined iteration limits.

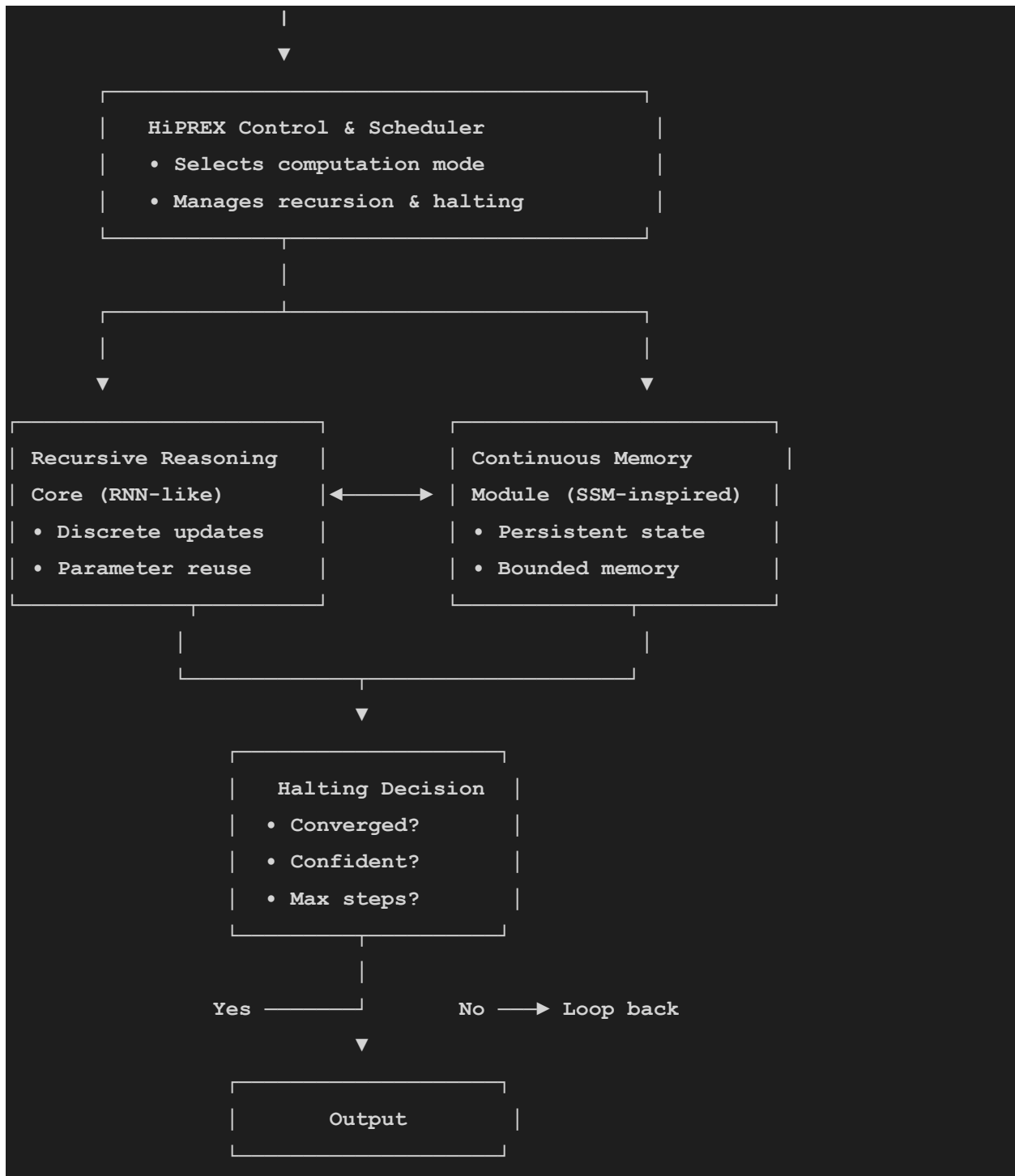
By externalizing control, HiPREX avoids the implicit assumption that all reasoning must be completed in a single static computation graph. Instead, computation is dynamically allocated based on task demands, allowing the system to adaptively balance global reasoning, discrete updates, and continuous memory.

2.6 Summary

HiPREX introduces a hierarchical architecture that integrates multiple neural computation paradigms under a unified control framework. Rather than favoring a single model class, the system leverages transformers for global semantic understanding, recursive models for discrete iterative reasoning, and state-based representations for persistent memory. By explicitly controlling when each paradigm is employed, HiPREX achieves memory-efficient inference while retaining expressive and flexible reasoning capabilities.

Figure 1: HiPREX Hierarchical Execution Flow





Section 3: Execution Flow in HiPREX

3.1 Overview of the Execution Pipeline

HiPREX executes inference as a staged process rather than a single monolithic computation. Instead of repeatedly invoking a large model, the system progresses through clearly defined phases: semantic initialization, recursive refinement, and termination. Each phase corresponds to a distinct mode of computation and is explicitly controlled by the HiPREX scheduler.

This section illustrates the execution flow using a representative example, highlighting how computation is distributed across different neural paradigms and how memory usage remains bounded throughout inference.

3.2 Semantic Initialization (Global Reasoning)

Given a user query, the first stage of execution invokes the **Semantic Initializer**, implemented using a transformer-based large language model. The purpose of this stage is to perform **global contextual reasoning** over the input.

For example, consider the following user query:

“Analyze why a basketball team’s offensive efficiency declined after the 2021 season.”

At this stage, the Semantic Initializer:

- Interprets the user’s intent
- Identifies relevant concepts (e.g., offensive efficiency, temporal comparison, contributing factors)
- Determines the scope of analysis
- Produces a structured internal representation encoding goals, constraints, and relevant variables

The output of this step is **not** a final answer, but an initial state that captures *what needs to be reasoned about*. Once this state is produced, the Semantic Initializer exits the inference loop and is not invoked again.

This design ensures that memory-intensive operations associated with global attention are performed exactly once.

3.3 Recursive Reasoning (Discrete State Updates)

The structured state produced during semantic initialization is then passed to the Recursive Reasoning Core. This component performs iterative refinement by repeatedly applying the same transformation function to the internal state.

At each iteration, the Recursive Reasoning Core:

- Examines the current state
- Identifies a local refinement or hypothesis update
- Produces a modified state reflecting incremental progress

In the example scenario, successive iterations may focus on:

- Comparing offensive metrics before and after 2021
- Evaluating roster or playstyle changes
- Refining hypotheses about causal factors

Importantly, each iteration reuses the **same parameters**, and no additional model weights or attention caches are introduced. As a result, memory usage remains constant across iterations, and extended reasoning is achieved through time rather than parameter scale.

This stage corresponds to **discrete reasoning**, analogous to recurrent neural updates or classical iterative algorithms.

3.4 Continuous Memory Integration

For tasks requiring persistent contextual information, HiPREX may incorporate a continuous memory component inspired by State Space Models (SSMs). This component maintains a compact, evolving representation of long-term state without relying on attention mechanisms. During recursive execution, the Recursive Reasoning Core may read from or write to this memory to:

- Preserve intermediate conclusions
- Track trends across iterations
- Maintain task-specific context

Because this memory is updated incrementally and does not grow with sequence length, it remains bounded in size and does not introduce the scaling issues associated with attention-based caches.

3.5 Halting Decision and Control

After each recursive iteration, the Halting and Control Mechanism evaluates whether further computation is necessary. Halting decisions may be based on criteria such as:

- Convergence of the internal state
- Stability of the proposed explanation
- Confidence thresholds
- Maximum iteration limits

If the halting condition is not satisfied, control is returned to the Recursive Reasoning Core for another iteration. Otherwise, the system proceeds to output generation.

By making halting explicit, HiPREX dynamically adapts computational effort to task complexity rather than enforcing a fixed computation depth.

3.6 Output Generation

Once a halting condition is met, the final internal state is converted into a user-facing response. In the example scenario, the system may output an explanation identifying specific factors, such as changes in shot distribution or roster composition, that contributed to the observed decline in offensive efficiency.

Because the Semantic Initializer is not re-invoked during this stage, output generation incurs minimal additional memory overhead.

3.7 Summary of Execution Flow

HiPREX executes inference as a controlled, multi-stage process:

1. A transformer-based model performs one-time global semantic reasoning.
2. A lightweight recursive core performs iterative refinement through discrete updates.
3. Continuous memory supports persistent state without attention.
4. An explicit halting mechanism governs termination.
5. The final output is produced from the refined internal state.

This execution flow shifts the primary scaling axis from static memory consumption to temporal computation, enabling efficient reasoning under constrained memory budgets.

Section 4: Memory and Computational Complexity Analysis

4.1 Motivation

A central objective of the HiPREX architecture is to address the growing memory constraints associated with large-scale neural inference. While many contemporary systems focus on reducing compute cost or improving model accuracy, inference-time memory usage increasingly determines feasibility, latency, and deployment scope. This section analyzes the memory and computational complexity of HiPREX and contrasts it with conventional large language model inference.

4.2 Memory Complexity of Transformer-Based Inference

Transformer-based language models rely on self-attention mechanisms that require maintaining key–value (KV) caches across inference steps. For a model with $\mathcal{L}$ layers, $\mathcal{H}$ attention heads, and hidden dimension d , the memory required to store the KV cache grows linearly with the sequence length T .

Conceptually, inference-time memory usage can be expressed as:

$$O(T \cdot \mathcal{L} \cdot \mathcal{H} \cdot d)$$

As a result:

- Memory usage increases with prompt length and generation duration
- Long-context or multi-turn interactions accumulate persistent memory
- Inference requires large amounts of GPU-local memory to maintain acceptable latency

Importantly, this memory must remain resident in low-latency GPU memory. Offloading to CPU memory or distributing KV caches across devices introduces latency that significantly degrades performance. Consequently, inference scalability is often constrained by available GPU VRAM rather than raw computational throughput.

4.3 Memory Complexity of HiPREX

HiPREX fundamentally alters this scaling behavior by separating inference into two distinct phases: semantic initialization and recursive reasoning.

Semantic Initialization Phase

The transformer-based Semantic Initializer is executed once per input. During this phase, memory usage follows the same scaling behavior as conventional transformer inference. However, because this phase is non-recursive and short-lived, attention-related memory structures are released immediately after the initial state is produced.

Thus, the memory cost of this phase is:

$$O(L \cdot H \cdot d)$$

independent of the duration of subsequent reasoning.

Recursive Reasoning Phase

During recursive execution, HiPREX employs a lightweight model with a fixed number of parameters and a bounded internal state. At each iteration, the same transformation function is applied, and no attention-based memory structures are accumulated.

As a result:

- Memory usage remains constant across iterations
- No KV cache growth occurs
- Internal state size is independent of recursion depth

The memory complexity of the recursive phase is therefore:

$$O(1)$$

with respect to the number of reasoning steps.

Continuous memory components inspired by State Space Models also maintain a fixed-size representation, preserving bounded memory usage.

4.4 Computational Complexity Tradeoffs

HiPREX intentionally trades **space complexity for time complexity**. While transformer-only systems perform all reasoning in a single forward pass, HiPREX distributes computation across multiple recursive iterations.

Let N denote the number of recursive steps. The computational cost of HiPREX scales as:

$$O(N \cdot C_r)$$

where C_r is the cost of a single recursive update.

This tradeoff is advantageous in memory-constrained environments, as it allows additional reasoning effort to be allocated dynamically without increasing memory usage. Tasks that require minimal reasoning may terminate after few iterations, while more complex tasks can consume additional compute time as needed.

4.5 Implications for Deployment

By bounding inference-time memory usage, HiPREX enables:

- Deployment on memory-constrained GPUs
- Extended reasoning without large context windows
- Predictable memory costs regardless of interaction length
- Improved suitability for edge devices and real-time systems

These properties directly address the limitations discussed in Section 1 and demonstrate how hierarchical, recursive computation provides a practical alternative to parameter-centric scaling.

4.7 Summary

HiPREX achieves memory-efficient inference by decoupling global semantic reasoning from iterative refinement. Through one-time transformer invocation and bounded recursive execution, the architecture shifts scalability from memory to time. This design aligns with classical algorithmic principles and offers a viable path forward in environments where high-quality memory is the dominant constraint.

Section 5: Comparison to Existing Neural Architectures

5.1 Overview

HiPREX does not introduce a new neural primitive, nor does it attempt to replace existing architectures. Instead, it provides a system-level orchestration framework that integrates multiple neural paradigms under explicit control. This section compares HiPREX to transformer-based models, recurrent neural networks, and state space models, highlighting how HiPREX leverages the strengths of each while mitigating their respective limitations.

5.2 Comparison with Transformer-Based Models

Transformer architectures are the foundation of modern large language models and are highly effective at capturing global context through self-attention mechanisms. This capability enables rich semantic understanding and long-range dependency modeling.

However, transformers exhibit several limitations during inference:

- Memory usage grows linearly with sequence length due to attention key–value caching
- All reasoning is performed in a single static computation graph
- Extended reasoning requires large context windows rather than iterative refinement

HiPREX retains transformers exclusively for **global semantic initialization**, where their strengths are most valuable. By invoking transformer-based reasoning only once per task, HiPREX avoids repeated attention computation and prevents unbounded memory growth. Subsequent reasoning is delegated to lightweight recursive components, shifting complexity from space to time.

In this sense, HiPREX treats transformers as **semantic encoders**, rather than universal reasoning engines.

5.3 Comparison with Recurrent Neural Networks (RNNs)

Recurrent Neural Networks perform computation through discrete state updates, making them well-suited for sequential and iterative tasks. Their parameter reuse naturally results in bounded memory usage across time steps.

However, RNNs suffer from notable limitations:

- Difficulty capturing long-range dependencies
- Limited global context awareness
- Challenges in training stability and expressiveness

HiPREX addresses these limitations by positioning recursive computation **after** global semantic understanding has already occurred. Rather than relying on an RNN to infer high-level meaning from raw input, HiPREX provides the recursive core with a structured initial state produced by a transformer. This allows the recursive component to focus on refinement and reasoning rather than representation learning.

Thus, HiPREX leverages the strengths of recurrence without inheriting its representational weaknesses.

5.4 Comparison with State Space Models (SSMs)

State Space Models maintain continuous internal representations that evolve over time, offering an efficient mechanism for long-term memory without attention-based scaling. Recent SSM-based architectures have demonstrated strong performance in sequence modeling tasks while maintaining favorable memory characteristics.

Nevertheless, SSMs typically operate under a fixed computational regime and lack explicit mechanisms for dynamic control or halting. They also do not inherently distinguish between stages of semantic understanding and iterative reasoning.

HiPREX incorporates SSM-inspired components as **optional continuous memory modules**, rather than as standalone architectures. These modules provide persistent state where needed, while higher-level control logic determines when they are engaged. This allows HiPREX to benefit from continuous memory without committing the entire inference process to a single model class.

5.5 Relation to Hierarchical Reasoning Models (HRM)

Recent work on Hierarchical Reasoning Models (HRM) explores the use of hierarchical depth and recurrent computation to achieve deep reasoning with bounded memory. HRM architectures typically decompose reasoning into multiple levels, such as a high-level planning module and a low-level execution module, which interact through iterative convergence cycles. By reusing parameters over time and avoiding explicit chain-of-thought tokenization, HRM demonstrates that effective reasoning depth can be achieved without relying on large context windows or growing attention caches.

HiPREX is conceptually aligned with this line of work in its emphasis on temporal depth, bounded memory, and latent-state reasoning. Both approaches reject the assumption that all reasoning must be performed in a single forward pass and instead leverage iterative computation to refine internal representations. In this sense, HRM provides empirical support for the broader architectural motivation underlying HiPREX.

However, the two approaches differ fundamentally in scope and abstraction level. HRM proposes a specific end-to-end neural architecture in which hierarchical structure, control, and memory are learned jointly within a single model. In contrast, HiPREX operates at the system level, introducing an explicit execution framework that orchestrates heterogeneous computational paradigms. Rather than defining a particular neural model, HiPREX specifies *when* global reasoning, discrete recursion, or continuous memory mechanisms should be employed during inference.

As a result, HRM-like models can be viewed as potential instantiations of the recursive reasoning component within the HiPREX framework, rather than as competing approaches. HiPREX is therefore complementary to HRM, providing a higher-level control architecture capable of integrating HRM-style recursive reasoning alongside transformers, state space models, or other specialized modules.

5.6 Architectural Positioning of HiPREX

The defining contribution of HiPREX lies not in outperforming existing architectures individually, but in **coordinating them effectively**. HiPREX introduces explicit control over computational mode, determining when global attention, discrete recursion, or continuous memory is appropriate.

This separation enables:

- Efficient use of transformers for semantic understanding
- Recursive models for memory-bounded reasoning
- State-based memory for persistent context
- Adaptive halting based on task complexity

By externalizing control, HiPREX avoids the implicit assumption that a single architecture must handle all aspects of reasoning.

5.7 Summary

HiPREX differs fundamentally from existing neural architectures by operating at the system level rather than the model level. While transformers, RNNs, and state space models each offer distinct advantages, they are typically employed in isolation. HiPREX unifies these paradigms under a hierarchical control framework, enabling efficient, adaptive, and memory-conscious inference.

This comparison underscores that HiPREX is not a replacement for existing models, but a principled method for integrating them in a way that aligns with both algorithmic reasoning principles and modern hardware constraints.

Section 6: Limitations and Open Challenges

6.1 Dependence on Semantic Initialization Quality

HiPREX relies on the Semantic_INITIALIZER to produce a structured internal representation that accurately captures task intent, constraints, and relevant context. Errors or ambiguities introduced at this stage may propagate through the recursive reasoning process, limiting the effectiveness of subsequent refinement.

Because the recursive core operates on the provided state rather than raw input, it may be unable to recover from incorrect high-level interpretations. This places significant importance on the robustness and reliability of the semantic initialization phase.

6.2 Halting Criteria and Control Design

Determining appropriate halting conditions remains a non-trivial challenge. While explicit halting enables adaptive computation, poorly chosen criteria may result in premature termination or unnecessary iterations.

Designing halting mechanisms that balance computational efficiency with reasoning completeness requires careful consideration. Confidence estimation, convergence detection, and task-specific thresholds may need to be tailored to different application domains, complicating general-purpose deployment.

6.3 Expressiveness of the Recursive Reasoning Core

The Recursive Reasoning Core is intentionally lightweight to maintain bounded memory usage. However, this constraint may limit its expressive capacity, particularly for tasks requiring complex transformations or deep abstractions.

Identifying the appropriate model class, state representation, and update function for the recursive core presents an open design challenge. Overly simplistic models may fail to capture necessary structure, while more complex models risk eroding the memory efficiency that HiPREX seeks to preserve.

6.4 Integration of Continuous Memory

While state space inspired memory modules provide an efficient mechanism for maintaining persistent state, their integration introduces additional design complexity. Deciding what information should be stored, how it should be updated, and how it should interact with discrete recursive updates remains an open research question.

Moreover, ensuring that continuous memory representations remain interpretable and stable over long execution horizons is a non-trivial challenge.

6.6 Latency and Real-Time Constraints

While HiPREX reduces memory consumption, it may introduce increased latency due to iterative execution. For applications with strict real-time requirements, the tradeoff between temporal computation and response time must be carefully managed.

Balancing recursion depth, halting sensitivity, and performance constraints presents an additional system-level challenge, particularly in interactive settings.

6.7 Summary

HiPREX introduces a principled approach to memory-efficient inference through hierarchical and recursive computation. However, its effectiveness depends on the quality of semantic initialization, the design of halting mechanisms, and the expressive capacity of lightweight recursive components. Addressing these limitations will require further research into control strategies, training methodologies, and model design, and represents a promising direction for future work.

Section 7: Conclusion and Future Work

7.1 Conclusion

This report introduced **HiPREX**, a hierarchical inference architecture designed to address the growing memory constraints of modern neural systems. By decoupling global semantic understanding from iterative reasoning, HiPREX shifts the primary scaling axis from static model size to temporal computation. A transformer-based semantic initializer performs one-time global reasoning, while a lightweight recursive core carries out extended refinement using bounded memory.

Through architectural analysis and comparison to existing neural paradigms, this work demonstrates that memory-efficient inference can be achieved without sacrificing expressive capability. Rather than replacing transformers, recurrent models, or state space models, HiPREX integrates their complementary strengths under explicit control. This design aligns with classical algorithmic principles and offers a practical response to the limitations imposed by inference-time memory growth.

7.2 Future Work

Several directions for future research emerge from this work. First, the design of adaptive and robust halting mechanisms remains an open challenge. Developing principled methods for convergence detection and confidence estimation would significantly improve the efficiency and reliability of recursive inference.

Second, further exploration of training strategies for hierarchical systems is needed. Investigating modular, staged, or partially supervised training approaches may enable more effective coordination between semantic initialization and recursive reasoning components.

Third, empirical evaluation of HiPREX across diverse tasks and deployment environments represents an important next step. Benchmarking memory usage, latency, and reasoning quality against transformer-only baselines would provide concrete evidence of the architecture's practical benefits.

Finally, extending the control framework to dynamically select among multiple recursive or memory modules could further enhance adaptability. Such extensions would move HiPREX toward a more general-purpose system capable of allocating computation flexibly in response to task demands.